



GOVERNMENT POLYTECHNIC MUZAFFARPUR

(Under Department of Science, Technology & Technical
Education Bihar)



LAB MANUAL

JAVA PROGRAMMING

Course Code: P2418401

INDEX

S. No.	Laboratory Experiment / Practical Titles	Relevant COs Number(s)
1	(a) Setup a Java Programming development environment by using: • Command prompt (Classpath and path setup) • Any IDE (Eclipse, JCreator etc.) (b) Test the IDE Setup by implementing a small Program	CO-1
2	Write programs to demonstrate use of if statement, if else, nested if statement for decision making.	CO-1
3	Write programs to demonstrate use of for loops and do-while loops.	CO-1
4	Develop a program for implementation of: • Constructors • Parameterized constructor • Non parameterized constructor	CO-2
5	Develop a program demonstrating implementation of different String Class functions.	CO-2
6	Develop program for implementation of Arrays in Java.	CO-2
7	Develop programs to demonstrate implementation of overloading and overriding concepts.	CO-3
8	Develop programs for implementation of: • Single inheritance • Multiple Inheritance • Multilevel Inheritance • Hierarchical Inheritance	CO-3
9	Develop a program to implement a try-catch block.	CO-4
10	Develop a program to demonstrate implementation of I/O stream classes.	CO-5
11	Develop a program to demonstrate implementation of file stream classes.	CO-5

1. COURSE INFORMATION

Particular	Details
Course Code	2418401 (T2418401 / P2418401 / S2418401)
Course Title	Java Programming
Programme	Diploma in Computer Science & Engineering
Semester	IV
Pre-requisite	ICT Tools
Laboratory Instruction	04 Hours/Week
Progressive Lab Assessment (PLA)	20 Marks
End Laboratory Assessment (ELA)	30 Marks

Rationale: Java is a platform-independent, object-oriented programming language widely used for developing robust applications. This course develops the ability to use classes, objects, control structures, constructors, strings, arrays, inheritance, exception handling, multithreading concepts, and input/output and file streams through core Java.

2. COURSE OUTCOMES (COs)

CO	Course Outcome
CO1	Implement control structure in Java using the concept of Class and Object.
CO2	Develop programs using Constructor and String Class.
CO3	Apply the concept of inheritance for code reusability.
CO4	Implement Exception Handling and multithreading.
CO5	Develop programs for handling I/O and file streams.

3. CO-PO ARTICULATION MATRIX

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7
CO1	1	-	-	-	-	-	1
CO2	-	1	-	1	-	-	-
CO3	-	2	1	1	-	-	1
CO4	2	1	1	1	-	-	-
CO5	1	1	1	1	-	-	-

Legend: 3 = High, 2 = Medium, 1 = Low, - = No Mapping

4. GENERAL LABORATORY INSTRUCTIONS

1. Read the aim, theory and procedure before starting each practical.
2. Save Java source files using the same name as the public class.
3. Use meaningful class, method and variable names with proper indentation.
4. Compile programs using javac or an approved IDE and remove syntax errors before execution.
5. Execute programs with suitable inputs and verify the output logically.
6. Record the source code, output, result and interpretation in the laboratory journal.
7. Handle exceptions appropriately where the practical requires it.

8. Complete the viva-voce and obtain faculty verification for every practical.

5. PROGRESSIVE LAB ASSESSMENT RUBRIC - 20 MARKS

S.No.	Category	Criteria of Assessment	Marks
1	Programming and Execution	Implementation of code / algorithm	3
2		Code quality (readability, structure, comments)	3
3		Debugging and testing	2
4	Result and Documentation	Accuracy of output / results	3
5		Correctness of logic & approach	5
6		Completeness of lab record	2
7	Viva-Voce	Conceptual understanding	2
		TOTAL	20

Distribution: Programming and Execution = 8 Marks | Result and Documentation = 10 Marks | Viva-Voce = 2 Marks

6. COMMON PRACTICAL ASSESSMENT RECORD

Practical No.	Date	Programming & Execution (8)	Result & Documentation (10)	Viva-Voce (2)	Total (20)
1					

PRACTICAL 1: JAVA DEVELOPMENT ENVIRONMENT SETUP AND FIRST PROGRAM

1. Aim

Set up the Java programming development environment using command prompt or an IDE and test the setup by executing a small Java program.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

The Java Development Kit (JDK) contains the tools required to create and run Java programs. The javac compiler translates a .java source file into platform-independent bytecode stored in a .class file. The Java Virtual Machine (JVM) executes the bytecode, which enables platform independence. The PATH environment variable helps the operating system locate Java tools, while CLASSPATH identifies locations containing compiled classes or libraries. A basic Java program contains a class declaration and a main() method, which serves as the entry point of a standalone Java application. Java source code is case-sensitive and the public class name must match the source filename.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1	LSO 1.1: Install Java IDE and verify the Java programming environment.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - Hello Java

```
public class HelloJava {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
        System.out.println("Java environment is working successfully.");
    }
}
```

Output

```
Hello, Java!
Java environment is working successfully.
```

Program B - Command-line argument

```
public class CommandLineDemo {
    public static void main(String[] args) {
        System.out.println("First argument: " + args[0]);
    }
}
```

Output

```
Command:
java CommandLineDemo Java

First argument: Java
```

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.
2. Use correct capitalization because Java is case-sensitive.
3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.
5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S.N o.	Viva-Voce Question	R	U	A
1	State the full form of JDK.	✓		
2	State the purpose of javac.	✓		
3	Name the file extension of Java bytecode.	✓		
4	Differentiate JDK, JRE and JVM.		✓	
5	Explain the role of the main() method.		✓	
6	Discuss platform independence in Java.		✓	
7	Compile a Java source file using javac.			✓

8	Execute a compiled Java class using the java command.			✓
9	Use a command-line argument in a Java program.			✓

Legend: R = Remember | U = Understanding | A = Apply

PRACTICAL 2: DECISION-MAKING CONTROL STATEMENTS

1. Aim

Develop Java programs using if, if-else and nested if statements for decision making.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

Decision-making statements select different execution paths according to Boolean conditions. The if statement executes a block when its condition is true. The if-else statement provides two alternatives, while nested if statements permit decisions inside other decisions. Java supports relational operators such as <, >, <=, >=, == and !=, and logical operators such as &&, || and !. Conditions in Java must evaluate to boolean values. Correct ordering of conditions is important when testing ranges such as grades or multiple criteria.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1	LSO 2.1: Develop programs using decision-making control statements in Java.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - Even or Odd

```
import java.util.Scanner;
public class EvenOdd {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        if (n % 2 == 0)
```



```
        System.out.println("Even");
    else
        System.out.println("Odd");
    }
}
```

Output

```
Enter a number: 18
Even
```

Program B - Largest of Three using nested if

```
import java.util.Scanner;
public class LargestThree {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter three numbers: ");
        int a = sc.nextInt(), b = sc.nextInt(), c = sc.nextInt();
        int max;
        if (a > b) {
            if (a > c) max = a;
            else max = c;
        } else {
            if (b > c) max = b;
            else max = c;
        }
        System.out.println("Largest = " + max);
    }
}
```

Output

```
Enter three numbers: 12 25 18
Largest = 25
```

Program C - Grade using if-else ladder

```
import java.util.Scanner;
public class GradeDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter marks: ");
        int marks = sc.nextInt();
        if (marks >= 80) System.out.println("Grade A");
        else if (marks >= 70) System.out.println("Grade B");
        else if (marks >= 60) System.out.println("Grade C");
        else if (marks >= 50) System.out.println("Grade D");
        else System.out.println("Grade F");
    }
}
```

Output

```
Enter marks: 76
Grade B
```

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.
2. Use correct capitalization because Java is case-sensitive.
3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.
5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Name three decision-making statements in Java.	✓		
2	State the equality operator in Java.	✓		
3	State the logical AND operator.	✓		
4	Differentiate if and if-else.		✓	
5	Explain nested if statements.		✓	
6	Discuss the ordering of conditions in an if-else ladder.		✓	
7	Construct a condition to test an even number.			✓
8	Apply nested if to select the largest among three values.			✓
9	Design an if-else ladder for grade classification.			✓

Legend: R = Remember | U = Understanding | A = Apply

PRACTICAL 3: LOOP STATEMENTS: FOR AND DO-WHILE

1. Aim

Develop Java programs using for and do-while loops to solve iterative problems.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

Loop statements repeat a block of code. A for loop combines initialization, condition and update in a compact form and is useful when the number of iterations is known. A do-while loop tests its condition after executing the loop body, so it runs at least once. Java also supports while and enhanced for loops. Loop variables, termination conditions and updates must be chosen carefully to avoid infinite loops. Nested loops are commonly used for patterns, tables and multidimensional data.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1	LSO 4.1: Develop programs in Java using loop statements to solve iterative problems.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - Multiplication table using for

```
import java.util.Scanner;
public class TableDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
```

```
int n = sc.nextInt();
for (int i = 1; i <= 10; i++)
    System.out.println(n + " x " + i + " = " + (n * i));
}
```

Output

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Program B - Sum of digits using do-while

```
import java.util.Scanner;
public class SumDigits {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt(), sum = 0;
        do {
            sum += n % 10;
            n /= 10;
        } while (n != 0);
        System.out.println("Sum of digits = " + sum);
    }
}
```

Output

```
Enter a number: 1234
Sum of digits = 10
```

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.
2. Use correct capitalization because Java is case-sensitive.

3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.
5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Name the loop statements available in Java.	✓		
2	Define an infinite loop.	✓		
3	State the position of condition testing in a do-while loop.	✓		
4	Differentiate for and do-while loops.		✓	
5	Explain the three parts of a for loop.		✓	
6	Discuss the use of modulus in digit-processing loops.		✓	
7	Construct a for loop to print 1 to 10.			✓
8	Apply do-while to process digits of a number.			✓
9	Create a loop to generate a multiplication table.			✓

Legend: R = Remember | U = Understanding | A = Apply

PRACTICAL 4: CONSTRUCTORS: DEFAULT, PARAMETERIZED AND NON-PARAMETERIZED

1. Aim

Develop Java programs to demonstrate constructors, parameterized constructors and non-parameterized constructors.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

A constructor is a special member of a class used to initialize objects. It has the same name as the class and has no return type. Java provides a default constructor only when no constructor is explicitly written. A non-parameterized constructor is explicitly defined without parameters, while a parameterized constructor receives values during object creation. Constructor overloading allows more than one constructor with different parameter lists. The `this` keyword can refer to the current object and is often used to distinguish instance variables from parameters.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO2	LSO 5.1: Implement constructors in Java.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - Non-parameterized constructor

```

class Student {
    String name;
    int roll;
    Student() {
        name = "Aman";
        roll = 101;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll: " + roll);
    }
}

public class ConstructorDemo1 {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}

```

Output

```

Name: Aman
Roll: 101

```

Program B - Parameterized constructor

```

class Rectangle {
    int length, breadth;
    Rectangle(int length, int breadth) {
        this.length = length;
        this.breadth = breadth;
    }
    int area() {
        return length * breadth;
    }
}

public class ConstructorDemo2 {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(10, 5);
        System.out.println("Area = " + r.area());
    }
}

```

Output

```

Area = 50

```

Program C - Constructor overloading

```

class Box {
    int length, breadth, height;
    Box() {
        length = breadth = height = 1;
    }
    Box(int l, int b, int h) {

```

```

        length = l; breadth = b; height = h;
    }
    int volume() {
        return length * breadth * height;
    }
}
public class ConstructorOverload {
    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = new Box(2, 3, 4);
        System.out.println("Default volume = " + b1.volume());
        System.out.println("Parameterized volume = " + b2.volume());
    }
}

```

Output

```

Default volume = 1
Parameterized volume = 24

```

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.
2. Use correct capitalization because Java is case-sensitive.
3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.
5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Define a constructor.	✓		
2	State the naming rule for a constructor.	✓		
3	State the purpose of the this keyword.	✓		
4	Differentiate non-parameterized and parameterized constructors.		✓	
5	Explain constructor overloading.		✓	
6	Discuss the role of constructors in object initialization.		✓	
7	Create an object using a parameterized constructor.			✓
8	Apply this to initialize instance variables.			✓
9	Develop two overloaded constructors for a class.			✓

Legend: R = Remember | U = Understanding | A = Apply

PRACTICAL 5: STRING CLASS FUNCTIONS

1. Aim

Develop Java programs demonstrating different functions of the String class.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

Java String objects represent sequences of characters. String is a class in java.lang and String objects are immutable, meaning their contents cannot be changed after creation. Common methods include length(), charAt(), substring(), equals(), equalsIgnoreCase(), compareTo(), indexOf(), toUpperCase(), toLowerCase(), trim() and replace(). The + operator can concatenate strings. StringBuffer and StringBuilder provide mutable character sequences and are useful when repeated modifications are required.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO2	LSO 6.1: Use different functions available in Java String class.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - Common String methods

```
public class StringMethods {  
    public static void main(String[] args) {  
        String s = "Java Programming";  
        System.out.println("Length = " + s.length());  
        System.out.println("Character at 2 = " + s.charAt(2));  
    }  
}
```

```

        System.out.println("Uppercase = " + s.toUpperCase());
        System.out.println("Substring = " + s.substring(5));
        System.out.println("Index of P = " + s.indexOf('P'));
    }
}

```

Output

```

Length = 16
Character at 2 = v
Uppercase = JAVA PROGRAMMING
Substring = Programming
Index of P = 5

```

Program B - String comparison and replace

```

public class StringCompare {
    public static void main(String[] args) {
        String a = "Java";
        String b = "java";
        System.out.println("equals = " + a.equals(b));
        System.out.println("equalsIgnoreCase = " + a.equalsIgnoreCase(b));
        System.out.println("replace = " + a.replace('a', 'o'));
    }
}

```

Output

```

equals = false
equalsIgnoreCase = true
replace = Jovo

```

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.
2. Use correct capitalization because Java is case-sensitive.
3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.
5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S.N o.	Viva-Voce Question	R	U	A
1	Define a String in Java.	✓		

2	State the method used to obtain String length.	✓		
3	Name one mutable string class.	✓		
4	Explain String immutability.		✓	
5	Differentiate equals() and equalsIgnoreCase().		✓	
6	Discuss String and StringBuffer.		✓	
7	Apply substring() to extract part of a String.			✓
8	Use replace() to modify displayed text.			✓
9	Select a suitable method to locate a character in a String.			✓

Legend: R = Remember | U = Understanding | A = Apply

PRACTICAL 6: ARRAYS IN JAVA

1. Aim

Develop Java programs for implementation of single-dimensional and multidimensional arrays.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

An array is an object that stores a fixed number of values of the same type. Java array indexing starts from zero and the length field stores the number of elements. Arrays are created using the new operator or initializer syntax. A two-dimensional array is an array of arrays and is commonly used for matrices and tabular data. Java checks array bounds at runtime and throws `ArrayIndexOutOfBoundsException` when an invalid index is accessed. Enhanced for loops can simplify traversal when the index itself is not required.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO2	LSO 7.1: Implement single and multidimensional arrays in Java.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - One-dimensional array

```
public class ArraySum {  
    public static void main(String[] args) {  
        int[] a = {10, 20, 30, 40, 50};  
        int sum = 0;
```

```

    for (int x : a) sum += x;
    System.out.println("Sum = " + sum);
    System.out.println("Average = " + (sum / (double)a.length));
}
}

```

Output

```

Sum = 150
Average = 30.0

```

Program B - Matrix addition

```

public class MatrixAddition {
    public static void main(String[] args) {
        int[][] a = {{1,2},{3,4}};
        int[][] b = {{5,6},{7,8}};
        int[][] c = new int[2][2];
        for (int i=0;i<2;i++)
            for (int j=0;j<2;j++)
                c[i][j] = a[i][j] + b[i][j];
        for (int[] row : c) {
            for (int x : row) System.out.print(x + " ");
            System.out.println();
        }
    }
}

```

Output

```

6 8
10 12

```

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.
2. Use correct capitalization because Java is case-sensitive.
3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.
5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S.No	Viva-Voce Question	R	U	A
.				

1	Define an array in Java.	✓		
2	State the first valid array index.	✓		
3	State the array property used to obtain its size.	✓		
4	Differentiate one-dimensional and two-dimensional arrays.		✓	
5	Explain runtime array-bound checking in Java.		✓	
6	Discuss the enhanced for loop for array traversal.		✓	
7	Declare an integer array of five elements.			✓
8	Apply an enhanced for loop to calculate a sum.			✓
9	Develop logic for matrix addition.			✓

Legend: R = Remember | U = Understanding | A = Apply

PRACTICAL 7: METHOD OVERLOADING AND METHOD OVERRIDING

1. Aim

Develop Java programs to demonstrate method overloading and method overriding.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

Method overloading occurs when methods in the same class share the same name but have different parameter lists. It is resolved at compile time and represents compile-time polymorphism. Method overriding occurs when a subclass provides a new implementation of an inherited method with the same signature. It is resolved at runtime through dynamic method dispatch and represents runtime polymorphism. The `@Override` annotation helps the compiler verify that overriding is intentional. Return type alone cannot distinguish overloaded methods.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO3	LSO 8.1: Implement overloading and overriding concepts in Java.

5. Resources Required

S.N o.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - Method overloading

```
class Calculator {  
    int add(int a, int b) { return a + b; }  
}
```

```

double add(double a, double b) { return a + b; }
int add(int a, int b, int c) { return a + b + c; }
}
public class OverloadingDemo {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(10,20));
        System.out.println(c.add(2.5,3.5));
        System.out.println(c.add(1,2,3));
    }
}

```

Output

```

30
6.0
6

```

Program B - Method overriding

```

class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
public class OverridingDemo {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}

```

Output

```

Dog barks

```

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.

2. Use correct capitalization because Java is case-sensitive.
3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.
5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Define method overloading.	✓		
2	Define method overriding.	✓		
3	State the annotation commonly used with overriding.	✓		
4	Differentiate overloading and overriding.		✓	
5	Explain compile-time polymorphism.		✓	
6	Discuss dynamic method dispatch.		✓	
7	Create two overloaded methods named add().			✓
8	Override an inherited method in a subclass.			✓
9	Apply a superclass reference to a subclass object.			✓

Legend: R = Remember | U = Understanding | A = Apply

PRACTICAL 8: INHERITANCE: SINGLE, MULTIPLE, MULTILEVEL AND HIERARCHICAL

1. Aim

Develop Java programs demonstrating single, multiple, multilevel and hierarchical inheritance.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

Inheritance allows a class to acquire fields and methods of another class, improving code reusability. Java uses the extends keyword for class inheritance. Single inheritance involves one direct superclass, multilevel inheritance forms a chain of inheritance, and hierarchical inheritance uses one superclass for multiple subclasses. Java does not support multiple inheritance through classes because it can create ambiguity, but similar behavior is achieved by implementing multiple interfaces. The super keyword accesses superclass members and constructors. Private members are not directly accessible in subclasses.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO3	LSO 9.1: Implement code reusability using inheritance in Java.

5. Resources Required

S. No.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - Single inheritance

```
class Person {
    String name = "Ravi";
}
class Student extends Person {
    int roll = 101;
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll: " + roll);
    }
}
public class SingleInheritance {
    public static void main(String[] args) {
        new Student().display();
    }
}
```

Output

```
Name: Ravi
Roll: 101
```

Program B - Multilevel inheritance

```
class A {
    void showA() { System.out.println("Class A"); }
}
class B extends A {
    void showB() { System.out.println("Class B"); }
}
class C extends B {
    void showC() { System.out.println("Class C"); }
}
public class MultilevelInheritance {
    public static void main(String[] args) {
        C obj = new C();
        obj.showA(); obj.showB(); obj.showC();
    }
}
```

Output

```
Class A
Class B
Class C
```

Program C - Hierarchical inheritance

```
class Shape {
    void info() { System.out.println("This is a shape"); }
}
class Circle extends Shape {
    void circle() { System.out.println("Circle class"); }
}
class Rectangle extends Shape {
    void rectangle() { System.out.println("Rectangle class"); }
}
```

```
public class HierarchicalInheritance {
    public static void main(String[] args) {
        Circle c = new Circle();
        Rectangle r = new Rectangle();
        c.info(); c.circle();
        r.info(); r.rectangle();
    }
}
```

Output

```
This is a shape
Circle class
This is a shape
Rectangle class
```

Program D - Multiple inheritance through interfaces

```
interface Printable {
    void print();
}
interface Showable {
    void show();
}
class Demo implements Printable, Showable {
    public void print() { System.out.println("Printable"); }
    public void show() { System.out.println("Showable"); }
}
public class MultipleInterfaceDemo {
    public static void main(String[] args) {
        Demo d = new Demo();
        d.print();
        d.show();
    }
}
```

Output

```
Printable
Showable
```

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.
2. Use correct capitalization because Java is case-sensitive.

3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.
5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S.N o.	Viva-Voce Question	R	U	A
1	Define inheritance.	✓		
2	State the keyword used for class inheritance.	✓		
3	Name four inheritance forms discussed in the syllabus.	✓		
4	Differentiate single and multilevel inheritance.		✓	
5	Explain hierarchical inheritance.		✓	
6	Discuss multiple inheritance through interfaces.		✓	
7	Create a subclass using extends.			✓
8	Apply multilevel inheritance to a three-class chain.			✓
9	Implement two interfaces in one class.			✓

Legend: R = Remember | U = Understanding | A = Apply

PRACTICAL 9: EXCEPTION HANDLING USING TRY-CATCH

1. Aim

Develop a Java program to implement try-catch exception handling.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

An exception is an abnormal condition that interrupts normal program execution. Java distinguishes errors from exceptions and provides structured exception handling using try, catch, finally, throw and throws. Code that may generate an exception is placed in a try block, while a matching catch block handles the exception. Multiple catch blocks can handle different exception types. The finally block is generally used for cleanup and executes whether or not an exception occurs. Built-in exceptions include ArithmeticException, ArrayIndexOutOfBoundsException and NumberFormatException.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO4	LSO 10.1: Apply the concepts of errors and exception handling.

5. Resources Required

S.N o.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - Arithmetic exception

```
public class ExceptionDemo {  
    public static void main(String[] args) {
```

```
try {
    int a = 10, b = 0;
    int c = a / b;
    System.out.println(c);
} catch (ArithmeticException e) {
    System.out.println("Division by zero is not allowed.");
}
System.out.println("Program continues normally.");
}
```

Output

```
Division by zero is not allowed.
Program continues normally.
```

Program B - Multiple catch blocks

```
public class MultipleCatchDemo {
    public static void main(String[] args) {
        try {
            int[] a = {10, 20};
            System.out.println(a[5]);
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic error");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index");
        } finally {
            System.out.println("Finally block executed");
        }
    }
}
```

Output

```
Invalid array index
Finally block executed
```

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.
2. Use correct capitalization because Java is case-sensitive.
3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.

5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S. No .	Viva-Voce Question	R	U	A
1	Define an exception.	✓		
2	Name the block used to handle an exception.	✓		
3	State the purpose of finally.	✓		
4	Differentiate errors and exceptions.		✓	
5	Explain try-catch execution.		✓	
6	Discuss multiple catch blocks.		✓	
7	Handle division by zero using try-catch.			✓
8	Apply a catch block for invalid array index.			✓
9	Use finally for a cleanup message.			✓

Legend: R = Remember | U = Understanding | A = Apply

PRACTICAL 10: I/O STREAM CLASSES

1. Aim

Develop Java programs demonstrating implementation of input/output stream classes.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

Java input/output is based on streams, which represent a flow of data between a source and a destination. Byte streams use InputStream and OutputStream hierarchies and are suitable for binary data. Character streams use Reader and Writer hierarchies and are suitable for text. System.in is a standard input stream and System.out is a standard output stream. BufferedReader can read text efficiently, while InputStreamReader bridges byte streams and character streams. Proper stream closing is important to release system resources.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO5	LSO 11.1: Implement file handling using I/O stream concepts.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - BufferedReader input

```
import java.io.*;
public class BufferedInputDemo {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new
        InputStreamReader(System.in));
    }
}
```

```

        System.out.print("Enter your name: ");
        String name = br.readLine();
        System.out.print("Enter your age: ");
        int age = Integer.parseInt(br.readLine());
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

```

Output

```

Enter your name: Anjali
Enter your age: 19
Name: Anjali
Age: 19

```

Program B - Character output using PrintWriter

```

import java.io.*;
public class PrintWriterDemo {
    public static void main(String[] args) {
        PrintWriter pw = new PrintWriter(System.out, true);
        pw.println("Java I/O Stream");
        pw.println("Output generated using PrintWriter.");
    }
}

```

Output

```

Java I/O Stream
Output generated using PrintWriter.

```

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.
2. Use correct capitalization because Java is case-sensitive.
3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.
5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S.N o.	Viva-Voce Question	R	U	A
1	Define a stream in Java.	✓		

2	Name the base class for byte input streams.	✓		
3	Name one character input class.	✓		
4	Differentiate byte streams and character streams.		✓	
5	Explain BufferedReader.		✓	
6	Discuss the role of InputStreamReader.		✓	
7	Read a line using BufferedReader.			✓
8	Convert a text input to integer using Integer.parseInt().			✓
9	Use PrintWriter to display formatted text.			✓

Legend: R = Remember | U = Understanding | A = Apply

PRACTICAL 11: FILE STREAM CLASSES

1. Aim

Develop Java programs demonstrating implementation of file stream classes.

2. Practical Significance

This practical develops hands-on understanding of the corresponding Java concept and strengthens the ability to write, compile, execute, test and interpret Java programs in a structured manner.

3. Relevant Theory

File streams connect Java programs to files stored on secondary storage. `FileInputStream` and `FileOutputStream` perform byte-oriented file operations, while `FileReader` and `FileWriter` handle character-oriented text files. `BufferedReader` and `BufferedWriter` can improve efficiency for text operations. The `File` class represents file and directory path information and can create, inspect or manage files. File operations can throw `IOException` or `FileNotFoundException`, so exception handling or throws declarations are required. Closing or using try-with-resources ensures that data is flushed and resources are released.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO5	LSO 12.1: Implement file handling using file stream classes.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	General-purpose computer with 4 GB RAM / 500 GB storage or better
2	Java Development Kit	JDK 1.8 or above
3	IDE / Editor	Eclipse / JCreator / IntelliJ IDEA / VS Code or command prompt
4	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs, processing and outputs.
2. Design the class structure, variables and methods required for the problem.
3. Write the Java source code using proper indentation and naming conventions.
4. Save the source file with the appropriate class name and .java extension.
5. Compile the program and correct compilation errors, if any.
6. Execute the program using suitable test data and verify the output.
7. Record the code, output, result and observations in the laboratory journal.

7. Programs / Implementation

Program A - Write and read a text file

```
import java.io.*;
public class FileWriterReaderDemo {
    public static void main(String[] args) throws IOException {
        FileWriter fw = new FileWriter("student.txt");
        fw.write("Java Programming Lab\n");
    }
}
```

```
fw.write("Semester IV\n");
fw.close();

BufferedReader br = new BufferedReader(new FileReader("student.txt"));
String line;
while ((line = br.readLine()) != null)
    System.out.println(line);
br.close();
}
}
```

Output

Java Programming Lab
Semester IV

Program B - Byte file stream

```
import java.io.*;
public class ByteFileDemo {
    public static void main(String[] args) throws IOException {
        FileOutputStream fos = new FileOutputStream("data.txt");
        String text = "Core Java";
        fos.write(text.getBytes());
        fos.close();

        FileInputStream fis = new FileInputStream("data.txt");
        int ch;
        while ((ch = fis.read()) != -1)
            System.out.print((char)ch);
        fis.close();
    }
}
```

Output

Core Java

8. Result

The specified Java program(s) were written, compiled and executed successfully. The obtained output was verified according to the intended program logic.

9. Interpretation

The practical demonstrates the correct application of the relevant Java programming concept. Program execution and output verification help relate theoretical concepts to object-oriented implementation and runtime behavior.

10. Cautions

1. Match the public class name with the source filename.
2. Use correct capitalization because Java is case-sensitive.
3. Initialize variables and objects before use.
4. Close streams/resources after use where applicable.

5. Test the program with more than one appropriate input set.

11. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Name the class used for character file output.	✓		
2	Name the class used for byte file input.	✓		
3	State the purpose of close().	✓		
4	Differentiate FileReader and FileInputStream.		✓	
5	Explain the purpose of buffering in file I/O.		✓	
6	Discuss IOException in file operations.		✓	
7	Write text to a file using FileWriter.			✓
8	Read a text file using BufferedReader.			✓
9	Apply FileInputStream to display byte data as characters.			✓

Legend: R = Remember | U = Understanding | A = Apply

SUGGESTED LEARNING RESOURCES

S.No.	Title	Author(s)	Publisher / Edition
1	Programming with JAVA	E. Balagurusamy	McGraw Hill Education (India) Private Limited, 5th Edition
2	Java Complete Reference JAVA 2	Herbert Schildt	McGraw Hill Education (India) Private Limited, 10th Edition
3	Head First Java	Kathy Sierra & Bert Bates	O'Reilly Media, 1st Edition